Soccer Goalie Project

Complete Code Packet

Arduino UNO + SG92R Servo + Photoresistor + Processing Display

---

> This packet contains both the Arduino and Processing code for the soccer goalie project. The Arduino controls the hardware. The Processing sketch displays the animated game interface.

Hardware Pin Summary

Part	Connection
Potentiometer middle pin	A0
Photoresistor divider node	A1
LED 1	D3
LED 2	D4
LED 3	D5
SG92R servo signal	D9

> Photoresistor divider: `5V -- photoresistor -- A1 -- 10k resistor -- GND`. The photoresistor works best when placed inside a small tube or tunnel so it sees light from one direction.

Arduino Code

```
// Soccer Goalie Project
// A0 = potentiometer
// A1 = photoresistor
// D3, D4, D5 = LEDs
// D9 = SG92R servo signal
// No Servo.h used

const int potPin = A0;
const int lightPin = A1;

const int led1 = 3;
const int led2 = 4;
const int led3 = 5;

const int servoPin = 9;

int score = 0;
```

```

// Servo settings
int servoAngle = 90;
int servoLeft = 45;
int servoRight = 135;

int minPulse = 900;
int maxPulse = 2100;

// Photoresistor values
int lightBase = 0;
int lightValue = 0;
int lightChange = 0;

// Goal threshold values
int goalThreshold = 12;
int resetThreshold = 5;

bool goalBlocked = false;
bool readyForNextGoal = true;

unsigned long lastGoalTime = 0;
unsigned long goalCooldown = 1800;

// LED/game states
bool goalFlash = false;
unsigned long goalFlashStart = 0;

bool winMode = false;
unsigned long winStart = 0;

unsigned long lastSerial = 0;

void setup() {
  Serial.begin(9600);

  pinMode(servoPin, OUTPUT);

  pinMode(led1, OUTPUT);
  pinMode(led2, OUTPUT);
  pinMode(led3, OUTPUT);

  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);
  digitalWrite(led3, LOW);

  // Learn normal light level once at startup.
  // Keep photoresistor uncovered during this.
  int total = 0;

  for (int i = 0; i < 100; i++) {
    total += analogRead(lightPin);
    delay(5);
  }

  lightBase = total / 100;
  lightValue = lightBase;

  blinkStart();
```

```
}

void loop() {
  readPot();

  // Send servo pulse before sensor/LED logic
  moveServo(servoAngle);

  readLight();
  checkGoal();
  updateLEDs();
  sendData();

  // Send servo pulse again so it does not freeze
  moveServo(servoAngle);
}

void readPot() {
  int potValue = analogRead(potPin);

  // Reversed direction.
  // If servo turns wrong way, swap servoRight and servoLeft.
  servoAngle = map(potValue, 0, 1023, servoRight, servoLeft);
}

void moveServo(int angle) {
  int pulse = map(angle, 0, 180, minPulse, maxPulse);

  digitalWrite(servoPin, HIGH);
  delayMicroseconds(pulse);
  digitalWrite(servoPin, LOW);

  delay(8);
}

void readLight() {
  lightValue = analogRead(lightPin);

  // Fixed baseline. Works whether value rises or falls.
  lightChange = abs(lightValue - lightBase);
}

void checkGoal() {
  bool blockedNow = lightChange > goalThreshold;
  bool returnedToNormal = lightChange < resetThreshold;

  // Re-arm only when the sensor is back to normal
  if (returnedToNormal) {
    goalBlocked = false;
    readyForNextGoal = true;
  }

  // Count only if sensor is blocked, armed, cooldown is over, and not in win mode
  if (blockedNow &&
      readyForNextGoal &&
      !goalBlocked &&
      !winMode &&
      millis() - lastGoalTime > goalCooldown) {

```

```
    goalBlocked = true;
    readyForNextGoal = false;
    lastGoalTime = millis();

    score++;

    goalFlash = true;
    goalFlashStart = millis();

    if (score >= 3) {
      winMode = true;
      winStart = millis();
    }
  }
}

void updateLEDs() {
  if (winMode) {
    winLights();

    if (millis() - winStart > 2400) {
      winMode = false;
      score = 0;
      showScore();

      // After reset, require sensor to be normal again
      readyForNextGoal = false;
      goalBlocked = true;
    }

    return;
  }

  if (goalFlash) {
    flashGoal();

    if (millis() - goalFlashStart > 500) {
      goalFlash = false;
      showScore();
    }

    return;
  }

  showScore();
}

void showScore() {
  digitalWrite(led1, score >= 1 ? HIGH : LOW);
  digitalWrite(led2, score >= 2 ? HIGH : LOW);
  digitalWrite(led3, LOW);
}

void flashGoal() {
  bool on = (millis() / 80) % 2 == 0;

  digitalWrite(led1, on ? HIGH : LOW);
  digitalWrite(led2, on ? HIGH : LOW);
  digitalWrite(led3, on ? HIGH : LOW);
```

```
}

void winLights() {
  int stepNum = (millis() / 90) % 8;

  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);
  digitalWrite(led3, LOW);

  if (stepNum == 0) digitalWrite(led1, HIGH);
  if (stepNum == 1) digitalWrite(led2, HIGH);
  if (stepNum == 2) digitalWrite(led3, HIGH);
  if (stepNum == 3) digitalWrite(led2, HIGH);

  if (stepNum == 4 || stepNum == 6) {
    digitalWrite(led1, HIGH);
    digitalWrite(led2, HIGH);
    digitalWrite(led3, HIGH);
  }
}

void blinkStart() {
  digitalWrite(led1, HIGH);
  delay(80);

  digitalWrite(led2, HIGH);
  delay(80);

  digitalWrite(led3, HIGH);
  delay(80);

  digitalWrite(led1, LOW);
  digitalWrite(led2, LOW);
  digitalWrite(led3, LOW);
}

void sendData() {
  if (millis() - lastSerial < 120) {
    return;
  }

  lastSerial = millis();

  if (winMode) {
    Serial.print("WIN");
  } else if (goalFlash) {
    Serial.print("GOAL");
  } else {
    Serial.print("READY");
  }

  Serial.print(",");
  Serial.print(score);

  Serial.print(",");
  Serial.print(servoAngle);

  Serial.print(",");
  Serial.print(lightValue);
```

```

  Serial.print(",");
  Serial.print(lightBase);

  Serial.print(",");
  Serial.println(lightChange);
}
```

Listing 1: soccer_goalie_arduino.ino

Processing Code

```
import processing.serial.*;

Serial myPort;

// Arduino sends:
// STATUS,SCORE,ANGLE,LIGHT,BASE,CHANGE

String status = "READY";
int score = 0;
int angle = 90;
int lightValue = 0;
int lightBase = 0;
int lightChange = 0;

// These must match the Arduino servo range
int servoLeft = 45;
int servoRight = 135;

int oldScore = 0;
int lastGoalTime = -2000;

// Goal box position
float goalX;
float goalY;
float goalW;
float goalH;

// Pivot point for goalie
float pivotX;
float pivotY;

// Ball animation variables
float ballX;
float ballY;
float ballStartX;
float ballStartY;
float ballEndX;
float ballEndY;

float ballRadius = 22;
float ballStartR = 28;
float ballEndR = 12;

boolean ballMoving = false;
int ballStartTime = 0;
int ballDuration = 900;
int ballImpactTime = -1000;

void setup() {
  size(1100, 700);
  smooth();
  textAlign(CENTER, CENTER);

  println(Serial.list());

```

```
  // Change [0] to [1] or [2] if the wrong port opens
  String portName = Serial.list()[0];
  myPort = new Serial(this, portName, 9600);
  myPort.bufferUntil('\n');

  // Smaller goal so the rotating goalie fills more of it
  goalW = 430;
  goalH = 230;
  goalX = width / 2 - goalW / 2;
  goalY = 155;

  // Fixed pivot near the bottom-left of the goal
  pivotX = goalX + 38;
  pivotY = goalY + goalH - 12;

  resetBall();
}

void draw() {
  drawBackground();
  drawGoal();
  drawField();
  drawBall();
  drawTopBar();
  drawScoreLights();
  drawSensorBox();
  drawGoalie();
  drawMessages();
}

// Read data from Arduino
void serialEvent(Serial port) {
  String line = port.readStringUntil('\n');

  if (line == null) {
    return;
  }

  line = trim(line);
  String[] parts = split(line, ',');

  if (parts.length >= 6) {
    oldScore = score;

    status = parts[0];
    score = int(parts[1]);
    angle = int(parts[2]);
    lightValue = int(parts[3]);
    lightBase = int(parts[4]);
    lightChange = int(parts[5]);

    // Only play the ball animation when score increases
    if (score > oldScore) {
      lastGoalTime = millis();
      startBallAnimation();
    }

    // If score resets after a win, put ball back
    if (score < oldScore) {
```

```
      resetBall();
    }
  }
}

// Draws sky and grass
void drawBackground() {
  if (status.equals("WIN")) {
    background(40, 25, 90);
  } else if (millis() - lastGoalTime < 350) {
    background(150, 25, 25);
  } else {
    background(120, 200, 245);
  }

  noStroke();

  // Sky
  fill(150, 220, 255);
  rect(0, 0, width, 260);

  // Grass
  fill(70, 170, 80);
  rect(0, 440, width, 260);

  // Grass stripes
  for (int i = 0; i < 12; i++) {
    if (i % 2 == 0) {
      fill(65, 160, 75);
    } else {
      fill(85, 185, 90);
    }

    rect(0, 440 + i * 24, width, 24);
  }
}

// Draws the goal and net
void drawGoal() {
  noStroke();

  // Goal back
  fill(245);
  rect(goalX, goalY, goalW, goalH, 16);

  // Inside shadow
  fill(0, 25);
  rect(goalX + 10, goalY + 10, goalW - 20, goalH - 20, 12);

  // Net lines
  stroke(175, 195, 215);
  strokeWeight(1.4);

  for (float x = goalX + 18; x < goalX + goalW; x += 22) {
    line(x, goalY + 10, x, goalY + goalH - 10);
  }

  for (float y = goalY + 18; y < goalY + goalH; y += 20) {
    line(goalX + 10, y, goalX + goalW - 10, y);
```

```
  }

  // Goal frame
  stroke(255);
  strokeWeight(14);
  noFill();
  rect(goalX, goalY, goalW, goalH, 16);

  // Pivot dot
  noStroke();
  fill(220, 30, 30);
  ellipse(pivotX, pivotY, 14, 14);

  fill(255);
  ellipse(pivotX, pivotY, 6, 6);
}

// Draws field lines
void drawField() {
  stroke(255);
  strokeWeight(4);
  noFill();

  rect(100, 465, width - 200, 170);
  line(width / 2, 465, width / 2, 635);
  ellipse(width / 2, 550, 95, 95);
}

// Top scoreboard
void drawTopBar() {
  noStroke();

  fill(30, 60, 90, 220);
  rect(0, 0, width, 78);

  fill(255);
  textSize(30);
  text("SOCCER GOALIE GAME", width / 2, 26);

  textSize(18);
  text("Score: " + score + "/3     Status: " + status + "     Servo Angle: " + angle, width / 2, 56);
}

// Draws three score lights
void drawScoreLights() {
  float x0 = width / 2 - 90;
  float y = 108;

  for (int i = 0; i < 3; i++) {
    boolean lit = false;

    if (status.equals("WIN")) {
      lit = true;
    } else if (score > i) {
      lit = true;
    }

    noStroke();

```

```
    if (lit) {
      fill(255, 30, 30);
    } else {
      fill(90, 0, 0);
    }

    ellipse(x0 + i * 90, y, 42, 42);

    fill(255);
    textSize(18);
    text(i + 1, x0 + i * 90, y);
  }
}

// Shows photoresistor values
void drawSensorBox() {
  int x = width - 255;
  int y = 515;
  int w = 210;
  int h = 150;

  noStroke();

  fill(0, 145);
  rect(x, y, w, h, 16);

  fill(255);
  textSize(20);
  text("PHOTORESISTOR", x + w / 2, y + 24);

  textSize(16);
  text("Light: " + lightValue, x + w / 2, y + 56);
  text("Base: " + lightBase, x + w / 2, y + 83);
  text("Change: " + lightChange, x + w / 2, y + 110);

  fill(45);
  rect(x + 20, y + 126, w - 40, 12, 6);

  float bar = map(lightChange, 0, 80, 0, w - 40);
  bar = constrain(bar, 0, w - 40);

  fill(255, 70, 70);
  rect(x + 20, y + 126, bar, 12, 6);
}

// Draws the goalie rotating around his feet
void drawGoalie() {
  // If the visual direction is wrong, swap 88 and -8
  float theta = map(angle, servoLeft, servoRight, radians(88), radians(-8));

  pushMatrix();

  translate(pivotX, pivotY);
  rotate(theta);

  drawGoalieBody(0, 0, 0.90);

  popMatrix();
}
```

```

// Reset ball to idle location
void resetBall() {
  ballX = width * 0.58;
  ballY = height - 110;
  ballRadius = 24;
}

// Start magical ball animation
void startBallAnimation() {
  ballMoving = true;
  ballStartTime = millis();

  // Starts from the field/front
  ballStartX = width * 0.58;
  ballStartY = height - 110;

  // Ends inside the goal/net
  ballEndX = goalX + goalW * 0.72;
  ballEndY = goalY + goalH * 0.62;

  // Starts big and gets smaller going into net
  ballStartR = 28;
  ballEndR = 12;
}

// Draw the ball and animation
void drawBall() {
  if (ballMoving) {
    float t = (millis() - ballStartTime) / float(ballDuration);
    t = constrain(t, 0, 1);

    // Smooth motion
    float smoothT = t * t * (3 - 2 * t);

    ballX = lerp(ballStartX, ballEndX, smoothT);
    ballY = lerp(ballStartY, ballEndY, smoothT);
    ballRadius = lerp(ballStartR, ballEndR, smoothT);

    // Little arc
    ballY -= sin(smoothT * PI) * 28;

    // Magic spawn effect at beginning
    if (t < 0.18) {
      drawMagicSpawn(ballX, ballY, ballRadius, t / 0.18);
    }

    drawBallGlow(ballX, ballY, ballRadius);
    drawSoccerBall(ballX, ballY, ballRadius);

    if (t >= 1) {
      ballMoving = false;
      ballImpactTime = millis();
    }
  } else {
    drawSoccerBall(ballX, ballY, 24);
  }

  // Sparkles when ball reaches goal
```

```
  if (millis() - ballImpactTime < 250) {
    drawGoalSparkle(ballEndX, ballEndY);
  }
}

// Draws the soccer ball
void drawSoccerBall(float x, float y, float r) {
  pushMatrix();
  translate(x, y);

  // soft shadow
  noStroke();
  fill(0, 40);
  ellipse(0, r * 0.9, r * 1.6, r * 0.55);

  // white ball
  fill(255);
  stroke(0);
  strokeWeight(2);
  ellipse(0, 0, r * 2, r * 2);

  // center patch
  fill(0);
  noStroke();

  beginShape();
  for (int i = 0; i < 5; i++) {
    float a = -HALF_PI + TWO_PI * i / 5;
    vertex(cos(a) * r * 0.34, sin(a) * r * 0.34);
  }
  endShape(CLOSE);

  // panel lines
  stroke(0);
  strokeWeight(1.6);

  for (int i = 0; i < 5; i++) {
    float a = -HALF_PI + TWO_PI * i / 5;
    line(cos(a) * r * 0.34, sin(a) * r * 0.34,
         cos(a) * r * 0.88, sin(a) * r * 0.88);
  }

  popMatrix();
}

// Glow around ball
void drawBallGlow(float x, float y, float r) {
  noStroke();

  fill(255, 240, 120, 70);
  ellipse(x, y, r * 3.2, r * 3.2);

  fill(255, 255, 255, 40);
  ellipse(x, y, r * 4.2, r * 4.2);
}

// Spawn rings and sparkles
void drawMagicSpawn(float x, float y, float r, float t) {
  pushMatrix();
```

```
  translate(x, y);

  noFill();
  strokeWeight(3);

  stroke(255, 230, 120, 255 * (1 - t));
  ellipse(0, 0, r * 2.5 + t * 35, r * 2.5 + t * 35);

  stroke(140, 220, 255, 180 * (1 - t));
  ellipse(0, 0, r * 3.2 + t * 50, r * 3.2 + t * 50);

  for (int i = 0; i < 8; i++) {
    float a = TWO_PI * i / 8.0 + t * 2.0;
    float d = r * 1.8 + t * 18;

    float sx = cos(a) * d;
    float sy = sin(a) * d;

    noStroke();
    fill(255, 255, 180, 220 * (1 - t));
    ellipse(sx, sy, 5, 5);
  }

  popMatrix();
}

// Sparkles inside the net
void drawGoalSparkle(float x, float y) {
  float t = (millis() - ballImpactTime) / 250.0;
  t = constrain(t, 0, 1);

  pushMatrix();
  translate(x, y);

  for (int i = 0; i < 10; i++) {
    float a = TWO_PI * i / 10.0;
    float d = 10 + 28 * t;

    float sx = cos(a) * d;
    float sy = sin(a) * d;

    stroke(255, 240, 120, 255 * (1 - t));
    strokeWeight(2);
    line(0, 0, sx, sy);

    noStroke();
    fill(255, 255, 180, 220 * (1 - t));
    ellipse(sx, sy, 4, 4);
  }

  popMatrix();
}

// Draws the goalie body
void drawGoalieBody(float x, float y, float s) {
  pushMatrix();

  translate(x, y);
  scale(s);
```

```

  // Origin is the midpoint between the feet

  noStroke();
  fill(0, 60);
  ellipse(0, 6, 120, 18);

  // Legs
  stroke(235, 190, 150);
  strokeWeight(16);
  line(-18, 0, -12, -55);
  line(18, 0, 12, -55);

  // Socks
  stroke(245);
  strokeWeight(12);
  line(-18, 0, -20, -18);
  line(18, 0, 20, -18);

  // Shoes
  noStroke();
  fill(55, 95, 190);
  ellipse(-22, 5, 30, 14);
  ellipse(22, 5, 30, 14);

  // Shorts
  stroke(40);
  strokeWeight(3);
  fill(245);

  beginShape();
  vertex(-42, -58);
  vertex(42, -58);
  vertex(55, -38);
  vertex(24, -18);
  vertex(0, -42);
  vertex(-24, -18);
  vertex(-55, -38);
  endShape(CLOSE);

  // Body
  fill(108, 68, 180);

  beginShape();
  vertex(-56, -120);
  bezierVertex(-48, -150, -20, -165, 0, -160);
  bezierVertex(20, -165, 48, -150, 56, -120);
  vertex(45, -58);
  bezierVertex(18, -44, -18, -44, -45, -58);
  endShape(CLOSE);

  // Jersey stripes
  noStroke();
  fill(246, 210, 55);
  rect(-28, -154, 12, 96, 4);
  rect(-6, -158, 12, 104, 4);
  rect(16, -154, 12, 96, 4);

  // Neck
```

```
stroke(40);
strokeWeight(3);
fill(240, 200, 160);
rect(-9, -172, 18, 14, 5);

// Arms
stroke(235, 190, 150);
strokeWeight(14);
line(-34, -122, -78, -146);
line(34, -122, 78, -146);

// Gloves
noStroke();
fill(255, 120, 35);
ellipse(-94, -152, 36, 36);
ellipse(94, -152, 36, 36);

fill(246, 220, 70);
ellipse(-101, -156, 16, 16);
ellipse(101, -156, 16, 16);

// Head
stroke(40);
strokeWeight(3);
fill(240, 200, 160);
ellipse(0, -210, 90, 90);

// Ears
ellipse(-44, -210, 14, 20);
ellipse(44, -210, 14, 20);

// Hair
fill(235, 125, 25);

beginShape();
vertex(-42, -222);
bezierVertex(-40, -250, -18, -265, 5, -258);
bezierVertex(22, -270, 45, -250, 42, -222);
bezierVertex(28, -234, 14, -232, 0, -224);
bezierVertex(-14, -232, -28, -234, -42, -222);
endShape(CLOSE);

fill(210, 95, 18);

beginShape();
vertex(-28, -222);
bezierVertex(-12, -240, 10, -240, 28, -222);
bezierVertex(12, -228, -10, -228, -28, -222);
endShape(CLOSE);

// Eyes
noStroke();
fill(255);
ellipse(-16, -216, 16, 20);
ellipse(16, -216, 16, 20);

fill(20);
ellipse(-16, -214, 8, 10);
ellipse(16, -214, 8, 10);
```

```

  // Eyebrows
  stroke(80, 40, 20);
  strokeWeight(3);
  line(-25, -228, -8, -223);
  line(8, -223, 25, -228);

  // Smile
  noFill();
  stroke(130, 45, 45);
  strokeWeight(4);
  arc(0, -194, 28, 16, 0, PI);

  // Jersey number
  noStroke();
  fill(255, 245, 170);
  rect(-6, -116, 12, 36, 3);
  rect(-12, -84, 24, 8, 3);
  quad(-6, -116, 4, -116, 12, -126, 2, -126);

  popMatrix();
}

// Draws goal and win messages
void drawMessages() {
  if (status.equals("WIN")) {
    fill(255, 240, 0);
    textSize(80);
    text("YOU WIN!", width / 2, 132);

    textSize(28);
    text("3 GOALS - RESETTING", width / 2, 180);
  } else if (millis() - lastGoalTime < 650) {
    fill(255, 240, 0);
    textSize(82);
    text("GOAL!!!", width / 2, 132);
  }
}
```

Listing 2: soccer_goalie_processing.pde